《算法分析与设计》

实验报告

学　　号　2024112639

姓　　名　毛露涵

年　　级　2024 级

专　　业　软件工程

院　　系　计算机与人工智能学院

实验预习（40%）	上机实验（40%）	文档排版（20%）	总分	批阅人签字

2025-2026 学年第 2 学期

实验 2.3 m_Coloring 算法时间复杂性分析

1. 实验目的

（1）实验比较数据输入规模对算法执行时间的影响。

（2）实验比较不同算法对问题求解时间的影响。

（3）掌握算法时间效率的分析方法。

2. 实验任务

（1）阅读并理解实验 2.3 的源代码。

（2）分析实验 2.3 源代码中 mColoring 函数的时间复杂性。

（3）用 C++语言实现该算法，绘制不同的 m、n 组合时算法的时间复杂度曲线和运行时间曲线，并进行简要分析。

（4）撰写实验报告，实验报告内容包括实验目的、实验任务、实验环境、实验步骤、实验结果和实验总结等部分。

3. 实验环境

3.1 件环境

（1） 计算机：Legion R9000P ADR10H

（2） CPU: AMD Ryzen 9 8945HX with Radeon Graphics

（3） RAM：32GB

3.2 软件环境

（1） 操作系统：Windows11 庭中文版

（2） 开发工具：Visual Studio 2026

4. 实验步骤及结果

4.1 实验预习

1. 解空间树

m=3,n=3 的解空间树，共有 3 个顶点，每个顶点可以选择 3 个颜色

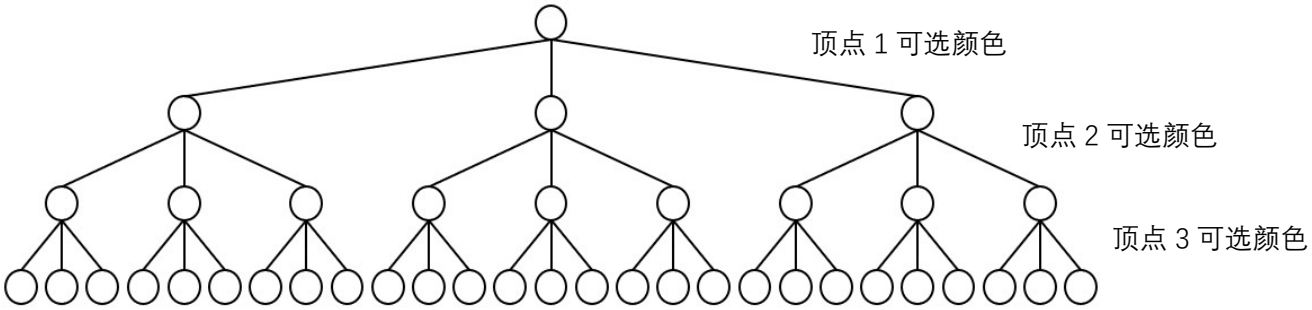


图 1 m=3,n=3 的解空间树

2. 时间复杂度递推公式

每轮的时间复杂度：T(t)=m·T(t+1)+m·n(t<=n)

递归出口的时间复杂度：T(t)=n(t>n)

3. 递推过程

$T(n)=m\cdot T(n+1)+m\cdot O(n)=m\cdot O(n)+m\cdot O(n)=O(m\cdot n)$

$T(n-1)=m\cdot T(n)+m\cdot O(n)=m\cdot O(mn)+m\cdot O(n)=O(m^2n+m\cdot n)$

$T(n-2)=m\cdot T(n-1)+m\cdot O(n)=O(m^3n+m^2n+m\cdot n)$

…

$T(1)=O(n\cdot(m^n+m^{n-1}+\cdots+m))=O(n\cdot m\cdot m^{n-1})=O(n\cdot m^n)$

4. 最坏时间复杂性

不考虑剪枝的情况下，算法的最坏时间复杂度为 $O(n\cdot m^n)$

4.1.1 用 C/C++语言实现的源代码

源代码：

```
#define _CRT_SECURE_NO_WARNINGS 1
//#include "stdafx.h"
#include "iostream"
#include <sys/timeb.h>
#include <stdlib.h>
#include <time.h>
#include <fstream>
using namespace std;

class Color {
    friend int mColoring(int, int, int**);
public:
    bool Ok(int k);
    void Backtrack(int t);
    int n, m, ** a, * x;
    long  sum;
};

int mColoring(int n, int m, int** a);

int main()
```

```
{
    const int TEST_MODE = 1;
    int RUN_TIMES = 10000;

    const int FIXED_M = 3;
    const int START_N = 3, END_N = 12, STEP_N = 1;

    const int FIXED_N = 10;
    const int START_M = 2, END_M = 6, STEP_M = 1;

    ofstream fout("coloring_time.csv");
    if (!fout.is_open()) {
        cout << "Cannot create output file!" << endl;
        return -1;
    }

    if (TEST_MODE == 1) {
        fout << "n,Total Time(ms),Avg Time(us)" << endl;
        cout << "===== Fixed m=" << FIXED_M << ", testing n (Empty Graph
- Full Search) =====" << endl;

        for (int n = START_N; n <= END_N; n += STEP_N) {
            if (n <= 8) RUN_TIMES = 10000;
            else if (n <= 10) RUN_TIMES = 1000;
            else RUN_TIMES = 100;

            long long totalTime = 0;

            int** a = new int* [n + 1];
            for (int i = 0; i <= n; i++) {
                a[i] = new int[n + 1];
                for (int j = 0; j <= n; j++)
                    a[i][j] = 0;
            }

            struct __timeb64 stime, etime;
            _ftime64(&stime);
            for (int run = 0; run < RUN_TIMES; run++) {
                mColoring(n, FIXED_M, a);
            }
            _ftime64(&etime);
            totalTime = (long long)((etime.time - stime.time) * 1000 +
(etime.millitm - stime.millitm));
```

```
            long long avgTimeUs = (totalTime * 1000) / RUN_TIMES;
            fout << n << "," << totalTime << "," << avgTimeUs << endl;
            cout << "n=" << n << ", Runs=" << RUN_TIMES << ", Total: "
<< totalTime << "ms, Avg: " << avgTimeUs << "us" << endl;

            for (int i = 0; i <= n; i++)
                delete[] a[i];
            delete[] a;
        }
    }
    else {
        fout << "m,Total Time(ms),Avg Time(us)" << endl;
        cout << "===== Fixed n=" << FIXED_N << ", testing m (Empty Graph
- Full Search) =====" << endl;

        int** a = new int* [FIXED_N + 1];
        for (int i = 0; i <= FIXED_N; i++) {
            a[i] = new int[FIXED_N + 1];
            for (int j = 0; j <= FIXED_N; j++)
                a[i][j] = 0;
        }

        for (int m = START_M; m <= END_M; m += STEP_M) {
            if (m <= 3) RUN_TIMES = 10000;
            else RUN_TIMES = 1000;

            long long totalTime = 0;

            struct __timeb64 stime, etime;
            _ftime64(&stime);
            for (int run = 0; run < RUN_TIMES; run++) {
                mColoring(FIXED_N, m, a);
            }
            _ftime64(&etime);
            totalTime = (long long)((etime.time - stime.time) * 1000 +
(etime.millitm - stime.millitm));

            long long avgTimeUs = (totalTime * 1000) / RUN_TIMES;
            fout << m << "," << totalTime << "," << avgTimeUs << endl;
            cout << "m=" << m << ", Runs=" << RUN_TIMES << ", Total: "
<< totalTime << "ms, Avg: " << avgTimeUs << "us" << endl;
        }

        for (int i = 0; i <= FIXED_N; i++)
```

```
            delete[] a[i];
        delete[] a;
    }

    fout.close();
    cout << endl << "Test finished! Data saved to coloring_time.csv" <<
endl;

    return 0;
}

bool Color::Ok(int k)
{
    for (int j = 1; j <= n; j++)
        if ((a[k][j] == 1) && (x[j] == x[k]))
            return false;
    return true;
}

void Color::Backtrack(int t)
{
    if (t > n) {
        sum++;
    }
    else {
        for (int i = 1; i <= m; i++) {
            x[t] = i;
            if (Ok(t))
                Backtrack(t + 1);
            x[t] = 0;
        }
    }
}

int mColoring(int n, int m, int** a)
{
    Color X;
    X.n = n;
    X.m = m;
    X.a = a;
    X.sum = 0;
    int* p = new int[n + 1];
    for (int i = 0; i <= n; i++)
        p[i] = 0;
```

```
    X.x = p;
    X.Backtrack(1);
    delete[] p;
    return X.sum;
}
```

4.2上机实验

4.2.1 m 不变 n 变化时的时间复杂度曲线和运行时间曲线

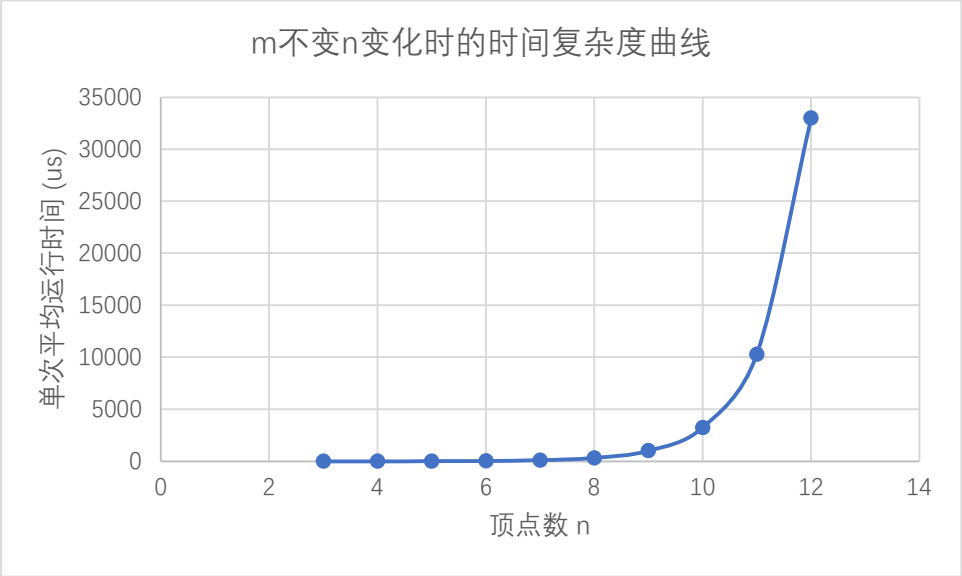


图 2 m 不变 n 变化时的时间复杂度曲线

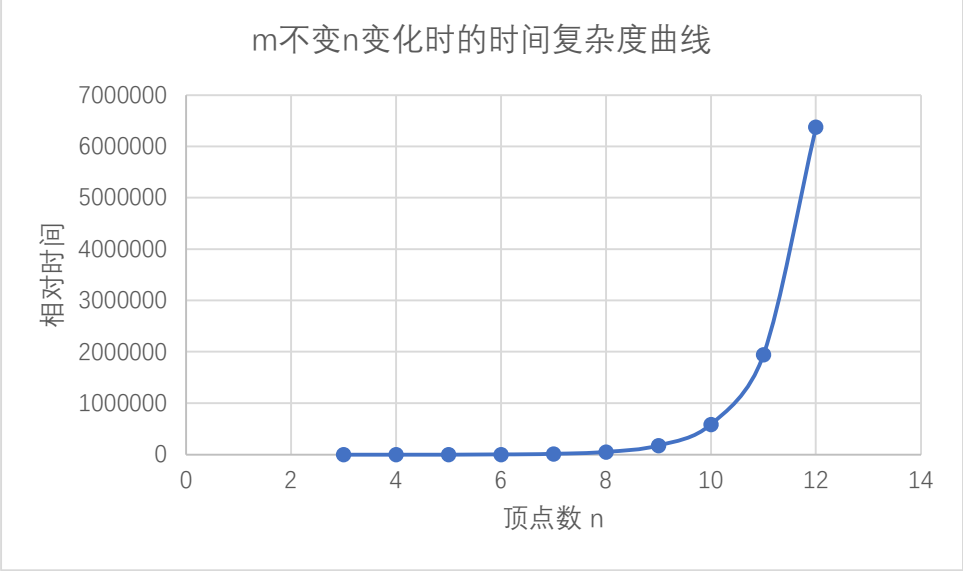


图 3 m 不变 n 变化时的时间复杂度曲线

4.2.2 n 不变 m 变化时的时间复杂度曲线和运行时间曲线

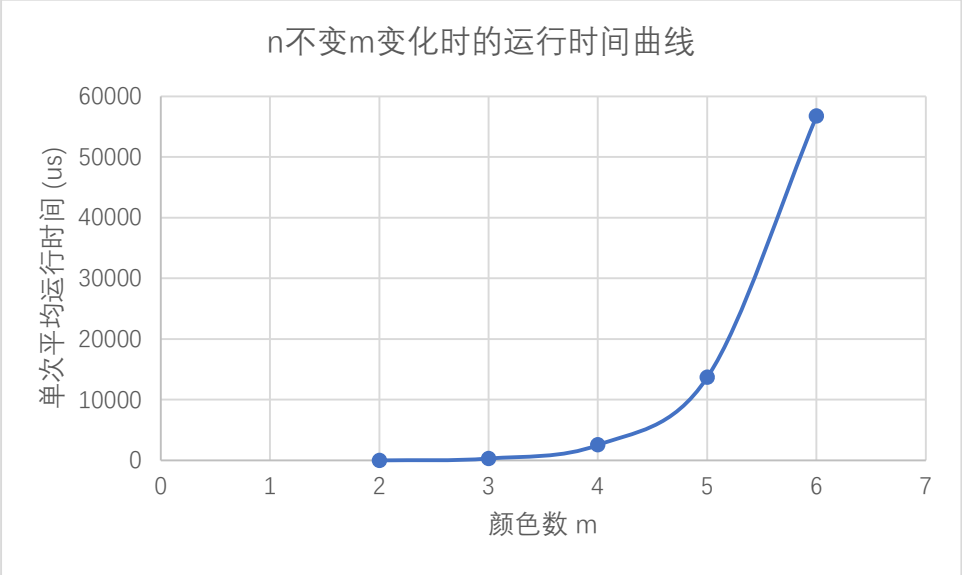


图 4 n 不变 m 变化时的运行时间曲线

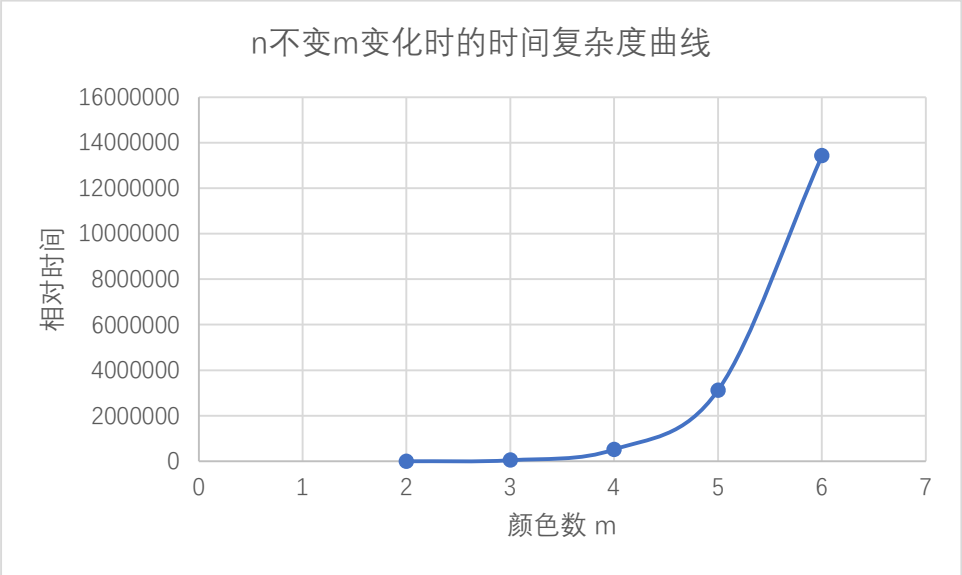


图 5 n 不变 m 变化时的时间复杂度曲线

4.2.3 m 在两种变化下，变量 n 的时间复杂度曲线和运行时间曲线

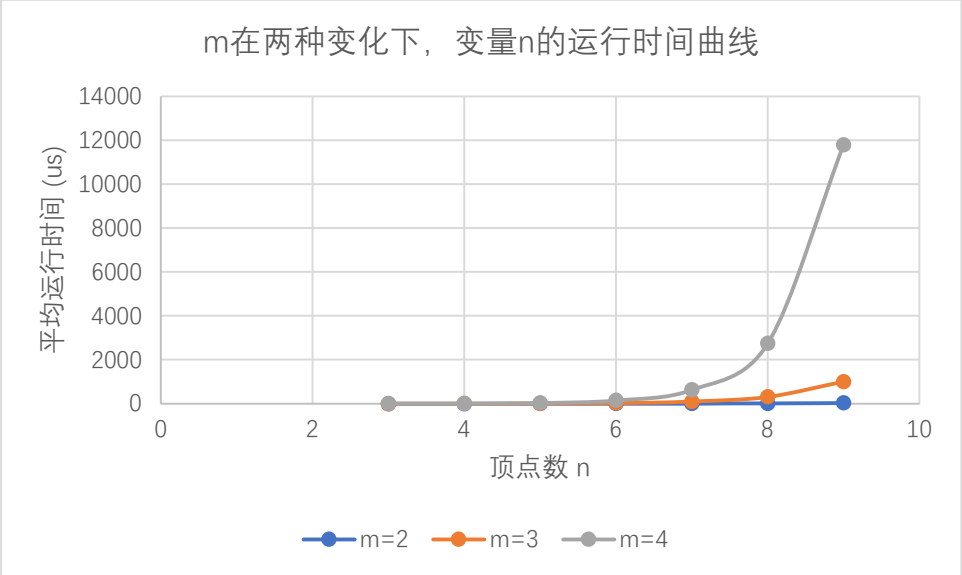


图 6 m 在两种变化下，变量 n 的运行时间曲线

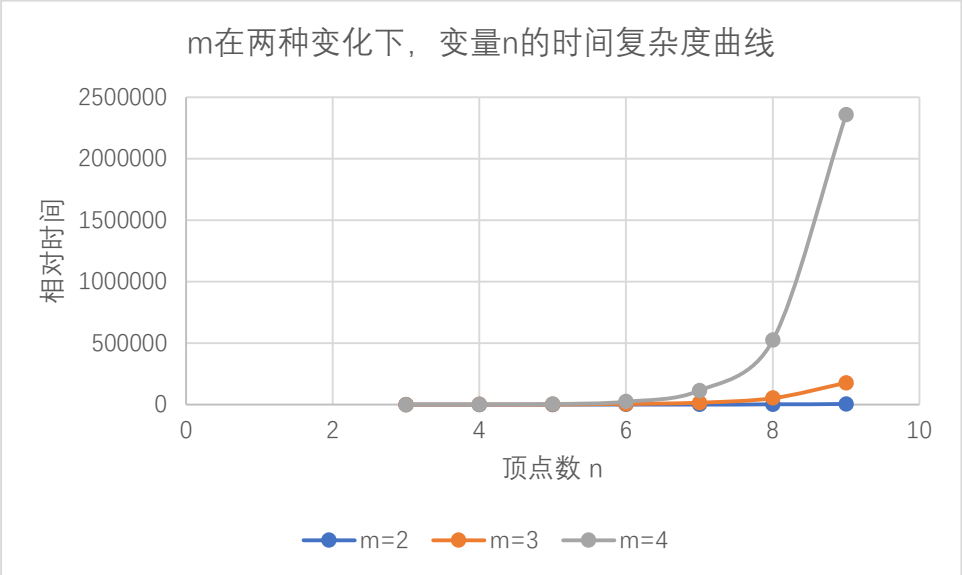


图 7 m 在两种变化下，变量 n 的时间复杂度曲线

5. 绘制 n=10 时例 2.5 代码的递归树

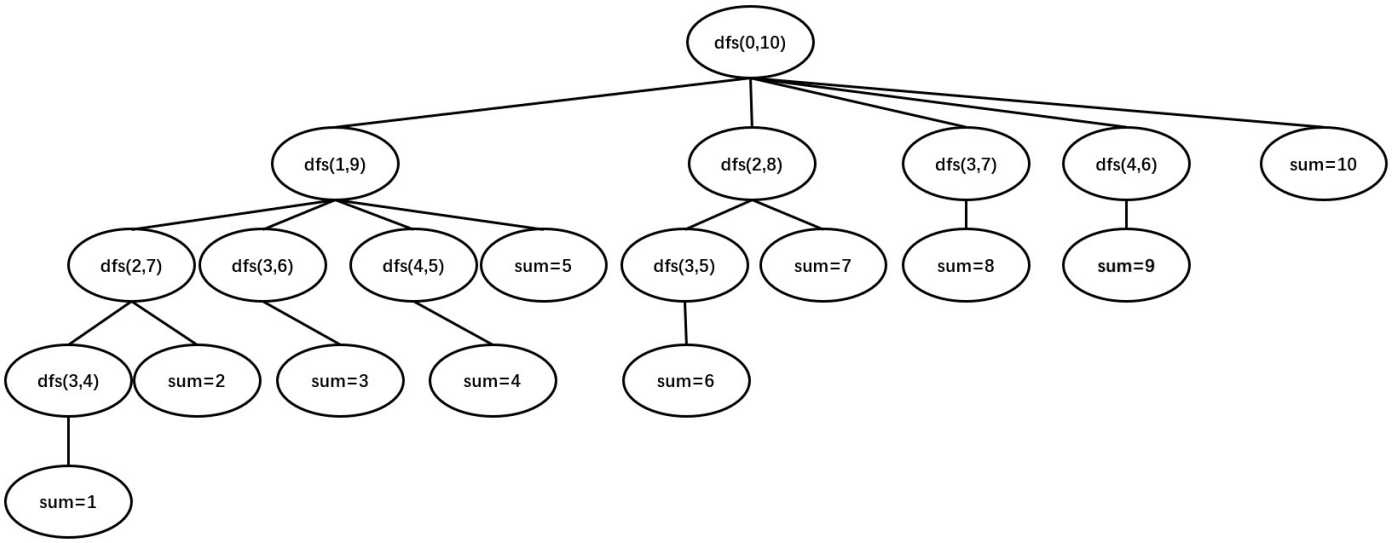


图 8 n=10 时例 2.5 代码的递归树

程序的输出 sum-1：9

6. 实验总结

本次实验完成了 m_Coloring 图着色回溯算法的时间复杂性分析，通过解空间树梳理与递推公式推导,得出算法在无剪枝全搜索下的最坏时间复杂度为$O(n \cdot m^n)$。实验用 C++实现了该算法，分别固定颜色数 m 测试顶点数 n、固定顶点数 n 测试颜色数 m，采集运行时间并绘制复杂度曲线，实验结果显示算法运行时间随 n 和 m 均呈指数级增长，与理论复杂度完全一致。实验中通过调整运行次数优化测试效率，规范内存管理避免了内存泄漏问题。通过本次实验，我熟练掌握了回溯算法时间复杂度的分析方法，将理论推导与实验验证相结合，提升了算法实现与性能测试的实践能力。